

Principles of Programming Languages

Type Systems

Andrei Arusoaie¹

¹Department of Computer Science

Outline

Motivation

Outline

Motivation

Typing

Outline

Motivation

Typing

Typing rules

Outline

Motivation

Typing

Typing rules

Properties

Motivation

Type errors in Coq:

```
(* Welcome to Coq 8.18.0 *)  
Coq < Check true + false.  
Toplevel input, characters 6-10:  
> Check true + false.  
>          ^^^^
```

Error:

The term "true" has type "bool" while
it **is** expected to **have** type "nat".

Motivation

... but in Javascript:

```
// Welcome to Node.js v18.16.0.  
// Type ".help" for more information.  
> true + false  
1  
> 2 + '4'  
'24'  
> '2' + 4  
'24'  
> 2 * '4'  
8  
> '2' * 4  
8
```

Motivation

What is the result type of `&&` in Javascript?

```
// Welcome to Node.js v18.16.0.  
// Type ".help" for more information.  
> 0 && true  
0  
> true && 0  
0  
> 0 && false  
0  
> false && 0  
false
```


Motivation

Confusing outputs in Javascript:

```
// Welcome to Node.js v18.16.0.  
// Type ".help" for more information.  
> false == []  
true  
> false == ![]  
true  
> [] == ![]  
true
```

Strongly vs. Weakly typed languages

- ▶ Coq is an example of a strongly-typed language
- ▶ Javascript is an example of a weakly-typed language

Strongly vs. Weakly typed languages

- ▶ Coq is an example of a strongly-typed language
- ▶ Javascript is an example of a weakly-typed language
- ▶ Strongly-typed languages: type safety, readability, performance vs. less flexibility, steep learning curve

Strongly vs. Weakly typed languages

- ▶ Coq is an example of a strongly-typed language
- ▶ Javascript is an example of a weakly-typed language
- ▶ Strongly-typed languages: type safety, readability, performance vs. less flexibility, steep learning curve
- ▶ Weakly-typed languages: flexibility, easy to learn, less verbose code vs. runtime errors, no type safety, possible less performance, debugging troubles

Goal for this lecture

Our goal in this lecture is to understand how to define a type system and what are its desired properties.

Steps:

- ▶ We first mix the arithmetic and boolean expressions
- ▶ We define the small-step SOS for such expressions
- ▶ We define a type system for these expressions
- ▶ We formulate and prove the following properties for our type system: *progress*, *preservation*, *soundness*

Mixing boolean and arithmetic expressions

Inductive Exp :=
| anum : nat -> Exp
| avar : string -> Exp
| aplus : Exp -> Exp -> Exp
| amult : Exp -> Exp -> Exp
| btrue : Exp
| bfalse : Exp
| bnot : Exp -> Exp
| band : Exp -> Exp -> Exp
| blessthan : Exp -> Exp -> Exp.

Coercion anum : nat >-> Exp.

Coercion avar : string >-> Exp.

Notation "A +' B" := (aplus A B) (at level 50,
left associativity).

Notation "A *' B" := (amult A B) (at level 40,
left associativity).

Notation "A <' B" := (blessthan A B) (at level 60).

Notation "A &&' B" := (band A B) (at level 71,
left associativity).

Let us mix our expressions in Coq

```
Eval compute in (2 +' 2) .  
= 2 +' 2  
: Exp
```

```
Eval compute in (2 +' btrue) .  
= 2 +' btrue  
: Exp
```

```
Eval compute in (band 2 2) .  
= band 2 2  
: Exp
```

Small-Step SOS for mixed expressions

Reserved Notation "A -[S]-> B" (at level 90).

Inductive eval : Exp -> Env -> Exp -> **Prop** :=

```
| const : forall i sigma,  
  (anum i) -[ sigma ]-> i  
| lookup : forall x sigma,  
  (avar x) -[ sigma ]-> (sigma x)  
| add_l : forall a1 a2 sigma a1',  
  a1 -[ sigma ]-> a1' ->  
  (a1 +' a2) -[ sigma ]-> (a1' +' a2)  
| add_r : forall a1 a2 sigma a2',  
  a2 -[ sigma ]-> a2' ->  
  (a1 +' a2) -[ sigma ]-> (a1 +' a2')  
| add : forall i1 i2 sigma n,  
  n = i1 + i2 ->  
  (i1 +' i2) -[ sigma ]-> n  
...  
| band_l : forall b1 b2 b1' sigma,  
  b1 -[ sigma ]-> b1' ->  
  (band b1 b2) -[ sigma ]-> (band b1' b2)  
| band_true : forall b2 sigma,  
  (band btrue b2) -[ sigma ]-> b2  
| band_false : forall b2 sigma,  
  (band bfalse b2) -[ sigma ]-> b2  
where "A -[ S ]-> B" := (eval A S B).
```


Well-typed expressions

Reserved Notation "A -[S]>* B"
(at level 90).

Inductive eval_closure :
Exp -> Env -> Exp -> **Prop** :=
| refl : **forall** e sigma,
 e -[sigma]>* e
| tran : **forall** e1 e2 e3 sigma,
 e1 -[sigma]-> e2 ->
 e2 -[sigma]>* e3 ->
 e1 -[sigma]>* e3
where "A -[S]>* B" :=
 (eval_closure A S B).

Example e1:
2 + ' "n" -[Env0]>* 2.

Proof.
 apply tran **with** (e2 := 2 + ' 0).
- **apply** add_r.
 apply lookup.
- **eapply** tran.
 + **apply** add. **trivial**.
 + **simpl.** **apply** refl.

Qed.

► Note: the expression 2 + ' "n" is evaluated to a **value**.

Ill-typed expressions

Example `ill_typed:`

```
2 +' btrue -[ Env0 ]>* ?.
```

- ▶ It is impossible to obtain a **value** using `eval`;
- ▶ In such cases we say that `2 +' btrue` is **stuck**:
`2 +' btrue` is **does not evaluation to a value** and no transitions via `eval` are possible from it

Adding types

- ▶ Types: Bool and Nat

- ▶ Coq:

```
Inductive Typ : Type :=  
  | Bool : Typ  
  | Nat : Typ.
```

- ▶ We define a relation $: \subseteq \text{Exp} \times \text{Typ}$ using inference rules for typing
- ▶ When $e : \text{Nat}$ we say that the type of e is Nat

Typing rules - I

$$\frac{n \in \mathbb{N}}{\text{anum } n : \text{Nat}} \text{ TNUM}$$

$$\frac{x \in \text{Var}}{\text{avar } x : \text{Nat}} \text{ TVAR}$$

$$\frac{a_1 : \text{Nat} \quad a_2 : \text{Nat}}{a_1 +' a_2 : \text{Nat}} \text{ TPLUS}$$

$$\frac{a_1 : \text{Nat} \quad a_2 : \text{Nat}}{a_1 *' a_2 : \text{Nat}} \text{ TMUL}$$

Typing rules - example

$$\frac{\frac{2 \in \mathbb{N}}{2 : \text{Nat}} \text{ TNUM} \quad \frac{x \in \text{Var}}{x : \text{Nat}} \text{ TVAR}}{2 +' x : \text{Nat}} \text{ TPLUS}$$

Typing rules - II

$$\frac{\cdot}{btrue : Bool} \text{ TTRUE}$$

$$\frac{\cdot}{bfalse : Bool} \text{ TFALSE}$$

$$\frac{b : Bool}{bnot \ b : Bool} \text{ TNOT}$$

$$\frac{b_1 : Bool \quad b_2 : Bool}{band \ b_1 \ b_2 : Bool} \text{ TAND}$$

$$\frac{a_1 : Nat \quad a_2 : Nat}{a_1 <' a_2 : Bool} \text{ TLT}$$

Typing rules - example

$$\frac{\frac{2 \in \mathbb{N}}{2 : \text{Nat}} \text{ TNUM} \quad \frac{x \in \text{Var}}{x : \text{Nat}} \text{ TVAR}}{2 \leq' x : \text{Bool}} \text{ TLEQ}$$

Typing rules - I - in Coq

```
Inductive type_of : Exp -> Typ -> Prop :=  
| typ_num : forall n, type_of (anum n) Nat  
| typ_var : forall x, type_of (avar x) Nat  
| typ_plus : forall a1 a2,  
               (type_of a1 Nat) ->  
               (type_of a2 Nat) ->  
               (type_of (a1 +' a2) Nat)  
| typ_mul : forall a1 a2,  
               (type_of a1 Nat) ->  
               (type_of a2 Nat) ->  
               (type_of (a1 *' a2) Nat)  
  
...
```


Typing rules - II - in Coq

```
Inductive type_of : Exp -> Typ -> Prop :=  
  ...  
| typ_true : type_of btrue Bool  
| typ_false : type_of bfalse Bool  
| typ_lessthan : forall a1 a2,  
  (type_of a1 Nat) ->  
  (type_of a2 Nat) ->  
  (type_of (a1 <' a2) Bool)  
| typ_not : forall b,  
  (type_of b Bool) ->  
  (type_of (bnot b) Bool)  
| typ_and : forall b1 b2,  
  (type_of b1 Bool) ->  
  (type_of b2 Bool) ->  
  (type_of (band b1 b2) Bool) .
```

Values

```
Inductive nat_value : Exp -> Prop :=  
| nat_val : forall n, nat_value (anum n).
```

```
Inductive b_value : Exp -> Prop :=  
| b_true : b_value btrue  
| b_false : b_value bfalse.
```

```
Definition value (e : Exp) :=  
  nat_value e \/ b_value e.
```

Canonical forms

```
Lemma bool_canonical :  
  forall e, type_of e Bool ->  
    value e ->  
    b_value e.
```

```
Lemma nat_canonical :  
  forall e, type_of e Nat ->  
    value e ->  
    nat_value e.
```

Progress

Theorem progress :
 forall t T state,
 type_of t T ->
 value t \ / **exists** t', t =[state]=> t'.

Proof.

(By induction on type_of *)*

Admitted.

Type preservation

Theorem preservation :
 forall t T t' state,
 type_of t T ->
 t =[state]=> t' ->
 type_of t' T.

Proof.

(By induction on type_of *)*

Admitted.

Type soundness

Corollary soundness :
 forall t t' state T,
 type_of t T ->
 t =[state]>* t' ->
 value t' \ / **exists** t'', t' =[state]=> t''.

Proof.

(By induction on the steps relation *)*

Admitted.

Resources

1. Software Foundations - Volume 2, Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, Andrew Tolmach, Brent Yorgey

Section “*Type Systems*”:

[https://softwarefoundations.cis.upenn.edu/
current/plf-current/Types.html](https://softwarefoundations.cis.upenn.edu/current/plf-current/Types.html)